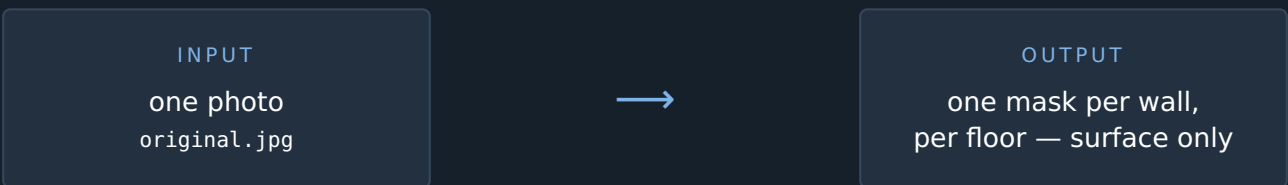


auto-wall-detect-matting

Wall and floor masks from one room photo — including the wall hidden behind the sofa.

We cannot segment what is hidden.
So we delete the furniture first, then segment the empty room.



WHAT IS INSIDE

1	What we are actually solving	the three requirements
2	The core idea	pipeline, stage by stage
3	Five models, five jobs	why each one is there
4	Why there are two detectors	the blind-spot argument
5	Running it	setup, notebook, CLI
6	When the result is wrong	symptom → knob
7	The variants are experiments	what each one proved
8	The matting stage	soft edges, and reading a metric
9	The other road: structure-first	no Imagen, no API
10	Training, in brief	Hypersim fine-tune
11	What to take away	transferable lessons

1 What we are actually solving

And why a normal segmentation model cannot do it.

The job: hand in a photo of a room, get back a clean mask for each wall and the floor. Downstream, someone repaints a wall or swaps the flooring. That single use case creates three hard requirements.

1. Surface only

RECALL BEATS PRECISION

One sofa pixel left inside the wall mask becomes a **sofa painted the new colour**. Missing an object is far worse than trimming a little real wall.

2. Include what is hidden

THE AMODAL REQUIREMENT

Repaint the wall, then move the sofa — the strip behind it must already be painted. This is the hard part, and the reason the pipeline looks the way it does.

3. One mask per wall

INSTANCES, NOT CLASSES

“Paint that wall” needs the left wall and the far wall as separate masks — not one merged wall class.

Bonus: soft edges

THE MATTING STAGE

Binary masks have stair-step edges and cannot represent a curtain fringe. A separate matting pass turns the hard mask into an alpha.

Why off-the-shelf semantic segmentation fails all three

AN ADE20K-STYLE WALL/FLOOR MODEL...	...BREAKS WHICH REQUIREMENT
labels the pixels behind the sofa as sofa	Req 2. You get visible wall only. No amodal output exists, at any confidence threshold.
has exactly one wall class	Req 3. Four walls come back as one connected blob. No instance separation.
emits hard, blocky boundaries	Bonus. Edge quality is a different problem needing a different model.

THE REFRAME

“Segment the wall behind the sofa” is not solvable by a segmenter — the pixels are simply not there. **“Remove the sofa, then segment” is solvable**, because it moves the impossible half of the task to a model that is genuinely good at it: a generative inpainter.

2 The core idea

Delete the furniture, then segment the empty room.

Detect every object, generatively inpaint it out of the photo, and run surface segmentation on the result. The segmenter now sees unobstructed wall, so its output is amodal **by construction** — no amodal-specific model, no mask-completion network, nothing to train.

THE PROPERTY THAT MAKES THIS SOUND

Imagen's `EDIT_MODE_INPAINT_REMOVAL` takes an explicit mask and rewrites **only the masked pixels**. Every other pixel is byte-identical to the input. So the empty room shares a pixel grid with the original photo, and a mask computed on the empty room applies to the original directly — **no homography, no re-alignment, no registration error**.

Once you have that, both deliverables fall out of one run. The expensive part is the amodal surface; the visible variant is a free subtraction.

```
amodal_surface = segment(empty_room)      ← the expensive part
visible_surface = amodal_surface - objects ← free
visible_floor   = amodal_floor  - objects - rug ← free
```

The pipeline, stage by stage

0 Kick off geometry MoGe · async, on the original image

Fired first, collected last, so it runs server-side while everything below runs locally. Pure latency hiding — costs roughly zero wall-clock instead of ~30 s.

→ [moge/](#)



1 Find everything that is not surface RAM++ → GroundingDINO → BiRefNet

RAM++ tags what is in this room. Structural words (wall, floor, ceiling, room...) are filtered out, and the rest becomes the **GroundingDINO** open-vocab prompt, which returns boxes. **BiRefNet** mattes the true silhouette inside each box (plus one full-image pass as a safety net). Boxes labelled rug/carpet/mat get their own mask.

→ [object_mask_birefnet.png](#) [rug_mask_birefnet.png](#)



2 Erase them Imagen 3 · inpaint removal

Send objects ∪ rug as the mask with an **empty prompt**. Imagen fills the holes with plausible wall and floor. Output is pixel-aligned to the input.

→ [empty_imagen.png](#) [inpaint_mask.png](#)



3 Segment the empty room image2scene API · auto_detect

Re-run surface detection on the emptied photo. Because the furniture is gone, the wall and floor masks it returns are **amodal** — and they arrive as a list, one per surface, which is where wall instances come from.

→ [wall_mask_amodal.png](#) [floor_mask_amodal.png](#) [from-api/](#)



4 Subtract to get the visible variant pure numpy · no model

Two boolean operations. Kept per instance, so each wall also gets its own visible mask, plus a colour overlay and per-stage timings.

→ [wall_mask_visible.png](#) [masked_images/wall_mask_visible_NN.png](#) [overlay.png](#) [time.txt](#)

3 Five models, five jobs

This is a pipeline of specialists, not one big model. Each stage is separately inspectable when a result goes wrong.

RAM++

NAMES WHAT IS IN THIS ROOM

GroundingDINO is open-vocab but **closed-set per call** — it only finds classes you name. A hardcoded list misses whatever this room happens to contain. RAM++ generates the vocabulary per image.

Also emits scene words — wall, floor, ceiling. Those must be filtered or you will erase the surface itself.

GroundingDINO

URNS NAMES INTO BOXES

RAM++ is a tagger: labels, no localization. Something has to convert tag → box. Class-agnostic NMS then removes the duplicate boxes that synonyms (sofa / couch / furniture) produce.

Prone to false positives on mirror ↔ window and framed art ↔ window. Thresholds need per-deployment tuning.

BiRefNet

PIXEL-ACCURATE SILHOUETTE

A box around a chair still contains wall. BiRefNet mattes the real shape inside each box — cropping per box is what gives small objects enough resolution to come out clean.

Structurally blind to flat wall-anchored objects. See section 4 — this is the single most important quirk in the repo.

Imagen 3

FILLS THE HOLES

This is what makes the output amodal. Its masked-inpaint contract (only masked pixels change) is what removes the need for any re-alignment step.

The only billed, remote, non-deterministic stage. Same input can give slightly different empty rooms — do not expect bit-reproducibility.

image2scene API

WALL / FLOOR INSTANCES

An internal service returning a **list** of per-wall masks rather than one merged class. Instance separation is a hard requirement, so this is why the list is decoded individually and the union derived from it — never the reverse.

MoGe

MONOCULAR GEOMETRY

Not used by the mask maths at all — collected for downstream 3-D consumers. Started at stage 0 and fetched at the end purely so its latency overlaps everything else.

PATTERN WORTH STEALING

Generate the vocabulary per input. Open-vocab detectors are closed-set per call, so a fixed class list quietly caps your recall. Putting a tagger in front turns a fixed list into an adaptive one. The repo goes one step further: plug and socket enter the prompt only when RAM++ actually tagged one — pay a class's false-positive cost only when there is evidence it is present.

4 Why there are two detectors

The most important non-obvious decision in the project.

BiRefNet is a salient 3-D foreground segmenter. It is excellent at sofas, tables, plants, boxes — anything that visually pops off the background. It is **structurally blind** to flat, wall-anchored items.

BiRefNet sees this

sofa · table · chair · plant · lamp · box · pet

Objects that sit in front of the wall. Clear depth cue, clear saliency, class-agnostic — it does not need to know what the thing is called.

BiRefNet cannot see this

window · mirror · poster · painting · wall-mounted TV · clock

Objects in the wall plane. No foreground/background depth cue, not salient. They stay in your wall mask, silently.

That is not a bug or a tuning failure — it is a direct consequence of what “salient foreground” means. No threshold fixes it. So a second detector is added whose entire job is to cover the first one's blind spot:

```
objects_all = BiRefNet( any 3-D foreground, class-agnostic )
               u GroundingDINO( named flat occluders )
```

“A surface without any object” is therefore **defined** as: any 3-D foreground object, plus any named flat occluder. Which makes the residual failure mode explicit and honest — **flat, unnamed wall items**. When you find a new one, you add it to the prompt vocabulary.

DESIGN RULE

Build the second detector around the first one's **blind spot**, not around its error rate. Two models that fail on the same inputs are one model with extra latency.

And why object anchoring never restricts subtraction

Objects can be described as wall-anchored (a mirror) or floor-anchored (a sofa). It is tempting to subtract only wall-anchored objects from the wall mask. **Don't.**

Tempting, wrong

`wall - wall_anchored_objects`

A sofa back, a tall plant, a bookshelf — all floor-anchored, all occluding the wall. Their pixels survive into the wall mask and get repainted.

Correct, and simpler

`wall - all_objects`

An object pixel is never legitimately part of any surface. Anchoring labels survive as QA metadata only.

Simpler and more correct — a rare combination.

5 Running it

Setup, data layout, and the two ways to invoke the pipeline.

Environment

```
conda create -n birefnet python=3.10 && conda activate birefnet
pip install -r requirements.txt
# install torch/torchvision for YOUR cuda: https://pytorch.org
gcloud auth application-default login # Imagen uses ADC, not an API key

# .env at the repo root:
GOOGLE_CLOUD_PROJECT=<your-gcp-project-id>
GOOGLE_CLOUD_LOCATION=us-central1
```

Weights and vendored model repos are **not in git** and must be fetched separately into `weights/` and `hq-mat/`: BiRefNet source + Hypersim fine-tunes, RAM++ checkpoint, SDMatte, DiffMatte, CLIP ViT-B/32.

Data layout

One folder per sample containing `original.jpg`. That is the only required input — everything else is written back beside it. Each pipeline variant has its own root so variants never overwrite each other.

```
users_gd/user_8/original.jpg      ← main_gd.ipynb reads this, writes everything else here
users_new_gd/ users_yolo/ ...     ← one root per variant
image2scene/                     ← run_api.py baseline (also the matting trimap source)
users_sd/ users_diff/            ← matting outputs
```

The notebook — recommended

Open `main_gd.ipynb` and run all cells. `load_models()` runs on import, then:

```
SAMPLE = "user_8"
res = run_pipeline(OUTPUT_DIR / SAMPLE, thresh=0.5)
```

`run_pipeline()` returns a dict holding every intermediate — boxes, labels, tags, prompt, mattes, empty room, masks, timings — which the cells below it consume. Keeping intermediates in memory is deliberate: tuning means looking at stage outputs, not at the final mask.

The three inspection cells, in the order you should use them

#	CELL	WHAT IT ANSWERS
1	All DINO boxes + labels, the RAM tag list, the assembled prompt	Were the right things even detected? Almost every bad result starts here.
2	Per-object BiRefNet matte grid — one tile per box, plus the full-image fallback	Detection was fine, so is the silhouette right inside each box?

3 3×3 panel: input, empty room, objects, rug, amodal + visible wall/floor, overlay	Did Imagen erase cleanly, and did the subtraction land?
---	---

Command line

```
python run.py --sample living_23          # one sample
python run.py --sample Bedroom --sample living_23  # several
python run.py --all                        # every folder with downsample.jpg
python run.py --all --thresh 0.4          # looser object threshold

python run_api.py --src users_gd --out image2scene --workers 5  # raw API baseline
```

COSTS MONEY

Every empty-room run bills **1 Imagen call + 1 image2scene job + 1 MoGe job per sample**. There is no dry-run mode, and the notebook's batch cell ships commented out on purpose. Test on one sample before any batch.

Two path quirks. `run.py` reads `downsample.jpg` from `output-test/`, while `main_gd.ipynb` reads `original.jpg` from `users_gd/` — historical drift, check which you have. And `run.py` looks for the RAM++ checkpoint at the repo root while the notebooks look in `weights/`; symlink if you use the CLI.

6 When the result is wrong

Symptom → cause → knob. Every knob lives at the top of the config cell.

SYMPTOM	CAUSE	FIX
Object left in the wall or floor mask	DINO never boxed it	Check the tag list in inspection cell 1. Lower <code>DINO_BOX_THRESH</code> , or add the class to <code>OBJECT_PROMPT</code> .
A flat wall item (poster, mirror) left in	BiRefNet is blind to it and DINO was not told to look	Add the class to the prompt. This is the known residual gap, not a tuning failure.
The wall itself got erased and inpainted	A structural RAM tag reached DINO	Add it to <code>RAM_EXCLUDE</code> / <code>RAM_EXCLUDE_KW</code> .
Halo or fringe around removed objects	Matte or inpaint mask too tight	Raise pad (<code>0.05</code> → <code>0.08</code>) or <code>mask_dilation</code> (<code>0.03</code> → <code>0.05</code>).
Imagen added furniture instead of removing it	<code>EMPTY_INPAINT_PROMPT</code> is not empty	Set it back to <code>"</code> ". See the callout below.
Sockets detected all over blank wall	DINO hallucinating small classes	That is what the <code>RAM_PLUG_KW</code> gate is for — confirm RAM++ actually tagged one.
Two real walls merged into one mask	The API returned a single instance	Upstream limitation — or use <code>main_normal.ipynb</code> , which splits by plane offset.
Mask edges blocky, soft boundaries missed	Binary mask, by design	Run the matting stage (section 8).
Missing key(s) ... bn ... on checkpoint load	<code>batch_size</code> config race	See the callout below.

GOTCHA 1 — THE PROMPT MUST BE EMPTY

Imagen reads the prompt as **what to generate in the masked region**. A helpful-sounding prompt like "empty room, bare floor" makes it **generate furniture** — it treats the text as a scene description, not an instruction. The empty string is what triggers clean content-aware removal. This is behavioural, not aesthetic.

GOTCHA 2 — THE ARCHITECTURE DEPENDS ON A GLOBAL CONFIG VALUE

BiRefNet swaps every BatchNorm for nn.Identity when config.batch_size ≤ 1. The fine-tune was trained at batch size 1, so if your kernel imported config.py while batch_size was still > 1, the model you build has BN layers the checkpoint does not — hence Missing key(s). The notebook forces the value on the relevant modules before constructing the net:

```
_aspp.config.batch_size = 1
_decblk.config.batch_size = 1
class _BS1Config(_bnmod.Config):
    def __init__(self, *a, **k):
        super().__init__(*a, **k); self.batch_size = 1
_bnmod.Config = _BS1Config
```

TWO SMALL DECISIONS THAT CARRY REAL WEIGHT

NEAREST, never bilinear, when resizing the API's binary masks — bilinear invents grey edge pixels that then threshold inconsistently. And **decode the mask list individually**, deriving the union from it rather than the other way round — that is the only thing preserving wall instances.

7 The variants are experiments

Every other notebook changes exactly one thing. Keeping them documents what was tried and why the main path won.

NOTEBOOK	CHANGES ONE THING	RESULT
main_gd.ipynb	— the baseline	Best. Use this one.
new_gd.ipynb	BiRefNet epoch_15	Checkpoint sweep
main_yolo.ipynb	YOLO-World + fixed class list, no RAM++	Faster, one model fewer — but fixed vocabulary
main_yolo_nano.ipynb	Gemini 2.5 Flash Image instead of Imagen	Loses pixel registration — see below
main_normal.ipynb	Drops Imagen and the API entirely	Fast, local, visible-only — section 9

The clearest lesson in the repo: mask-based vs prompt-based editing

Imagen — masked inpaint	Gemini “nano banana” — prompt edit
Takes a mask. Rewrites only the masked pixels. Everything else is byte-identical. Masks transfer back for free.	No mask channel at all. May move, recolour or re-render pixels outside the region you meant to change. Empty room no longer aligned with the original. Needs a homography estimate — which adds its own error.

TAKE-AWAY

The capability that mattered here was **not image quality** — it was the **guarantee about untouched pixels**. A mask-based editor gives you a contract; a prompt-based editor gives you a result. When picking a generative component, ask what it promises not to do.

RAM++ → DINO vs YOLO-World

YOLO-World is open-vocab with built-in NMS and is faster — one model instead of two. But its class list is **fixed at config time**, and rooms are long-tailed: the class you did not list is the one in this photo. RAM++ solves the vocabulary

problem per image, which is worth the extra model. Keep the YOLO variant for speed-constrained deployments where you control the furniture distribution.

8 The matting stage

Turning a hard binary mask into a soft alpha — and a lesson in how to read a metric.

Everything so far produces binary masks: stair-stepped edges, and no way to represent a curtain fringe or a plant frond. `main_sd.ipynb` and `main_diff.ipynb` refine a wall mask into an alpha.

Trimap discipline

Both notebooks **import the same function**, `generate_trimap.wall_trimap()`. That is what makes the comparison valid: identical input, so any difference isolates the network. A shared import cannot drift the way two copy-pasted preprocessing blocks will.

The trimap source is the raw API wall mask from the untouched photo — deliberately **not** the object-removed one. Matting must learn from real photographic edges; an inpainted edge is synthetic, and the matting network has no business seeing it.

	SDMATTE	DIFFMATTE
Backbone	Stable Diffusion U-Net, ~1.3 B params	ViT-S, 29 M + tiny diffusion decoder
Prompt types	trimap / mask / box / points	trimap only
Iterations	1 step, denoise in latent space	N steps, denoise the alpha directly
VRAM @1024 ²	~5 GB (fp16)	~1.6 GB (fp32)

The head-to-head — and how to read it

50 samples, shared trimap, alpha threshold 0.5.

0.981 whole-image IoU (nearly meaningless)	1.98% of pixels are actually ambiguous	0.740 band agreement the real number	45× smaller backbone for that result
--	--	--	--

READ THE METRIC WHERE THE METHOD ACTS

Whole-image IoU of 0.98 says almost nothing: 98% of pixels are definite foreground or background that the trimap already decided, so both networks trivially agree there. The informative number is **band agreement: 0.740** — inside the 2% of pixels that are genuinely ambiguous, the only pixels matting exists to resolve, the two networks disagree a quarter of the time.

Always define your metric over the region your method is responsible for. A whole-image average will hide the exact effect you are trying to measure.

Engineering conclusion: DiffMatte reaches that result with a 45× smaller backbone and a third of the VRAM. Unless the band disagreement resolves in SDMatte's favour on your data, the small model wins.

Practical notes

- SDMatte is built **config-only**, then `SDMatte.pth` is loaded on top. `weights_only=False` + `mmap=True` are required — the checkpoint pickles an `omegaconf.ListConfig`.
- DiffMatte needs no `detectron2`; a small `d2shim` supplies the few symbols it imports.
- SDMatte's `points` mode cannot encode negatives, and signed blobs collapse the alpha to zero. Trimap is the only mode that works reliably.
- `PROMPT_MODE="trimap"` skips RAM++ and DINO entirely — about 4 GB VRAM and 5 s/sample saved.
- EXIF-rotated samples are normalized **in memory**, never on disk.

9 The other road: structure-first

`main_normal.ipynb` — no Imagen, no API, no detector stack. A genuine trade, not a strict improvement.

Motivated by two real problems with the main path: Imagen is slow and billed, and it **ghosts** — hallucinating faint content into the emptied room, which the segmenter then faithfully segments.

1 Invert the object mask BiRefNet, fine-tuned epoch_10

Segment movable objects, then invert → the wall + floor + ceiling shell. Works because Hypersim's background class set teaches door, window, mirror and ceiling as **structure** — the flat-occluder blind spot from section 4 becomes an asset here.

2 Depth and normals in one pass Metric3D-v2

One feed-forward net giving per-pixel metric depth and surface normal. Chosen over Marigold precisely because it is feed-forward — consistent with the “kill Imagen for speed” motive.

3 Split the point cloud 30° cone → HDBSCAN → connected components

Floor / ceiling / wall by the angle between the normal and up — these are ~90° apart, so the threshold is not delicate. Then wall instances by HDBSCAN on `normal @ W_OFFSET · d`, then 2-D connected components for disjoint same-plane pieces.

Why depth is mandatory, not a nice-to-have

The motivating case is two **jagged sub-walls facing the same direction**. They have **identical normals**, and they can be 2-D connected. Neither normal angle nor connectivity can tell them apart. Only the **plane offset** can:

```
d = point · n          # signed distance along the shared normal
```

Two parallel planes at different depths have different d . That single geometric fact is why the pipeline needs a depth model rather than a normals-only one. `W_OFFSET` trades the two cues off: 0 is pure-normal (same-faced walls merge), higher splits more aggressively.

Scaling detail worth remembering: HDBSCAN is fit on a 30k-pixel subsample and all ~1M wall pixels are then assigned by **nearest cluster centroid** — `approximate_predict` at that scale is far too slow.

VERIFIED ON USER_2

Structure fraction **0.995** — correctly leaves an empty hallway as structure. Metric3D-v2 + HDBSCAN found 4 clusters and cleanly split left wall / right wall / far wall / floor / ceiling → 8 instance masks after connected components. The separation method is validated.

TWO LIMITS YOU ARE ACCEPTING

Visible-only. Inversion cannot recover what is behind furniture — every object leaves a hole, and the amodal output is simply not reproduced. That is the trade: speed and locality in exchange for amodality.

No safety net. A BiRefNet miss leaves an object as fake surface; a false positive punches a hole in the wall. Unlike the main path, there is no second detector to catch the first one's mistakes. Quality rides entirely on one model.

The checkpoint lesson

CHECKPOINT	STRUCTURE FRACTION ON AN EMPTY HALLWAY	VERDICT
epoch_10.pth	0.995	Correct. This is the one main_normal uses.
epoch_10_new.pth	0.34	Called 66% of a bare room “object”. Unusable here; the re-train was cancelled.

BUT NOTE THE ASYMMETRY

epoch_10_new is perfectly **fine in main_gd.ipynb**, where BiRefNet only mattes inside a DINO box and is never asked to judge empty wall. The same checkpoint is unusable in main_normal, where the whole image is its responsibility.

A checkpoint is good only relative to the job you give it. Always re-benchmark on the actual downstream task, never on the training metric. That 0.34 is also the honest measurement of the **synthetic→real gap**: more epochs on synthetic Hypersim data made the model worse on real photographs.

10 Training, in brief

Teaching BiRefNet that “foreground” means “everything that is not room structure”.

The stock model segments salient objects. The fine-tune redefines the foreground class. Data is **Hypersim** (Apple) — photorealistic synthetic indoor scenes, CC BY-SA 3.0, with pixel-perfect rendered semantic labels. Labels are free and exact, which is exactly why synthetic data is attractive here — and section 9 is the bill for it.

NOTEBOOK	METHOD
trainer.ipynb	Guided walkthrough of the first fine-tune: download, prep, train. Heavy steps are flag-guarded and off by default , so Run All stays fast — a good pattern for a notebook that doubles as documentation.
trainer_v2.ipynb	Proper methodology, below.

The v2 methodology — worth imitating

- **Split by SCENE, 70/15/15 — not by frame.** Hypersim gives many frames per room. A frame-level split puts near-duplicate views of the same room in both train and test and inflates your metrics enormously. Splitting by scene is the only honest option.
- **Phase A:** train on TRAIN, monitor VAL each epoch, early-stop on val mean-IoU (max 50 epochs), keep the best checkpoint.
- **Phase B:** once VAL has served model selection, fold it into training for a few more epochs. The data has done its job; now it can be signal.
- **TEST is looked at exactly once**, at the very end.
- Checkpoints and history.json are written **every epoch**, so an interrupted run is never a lost run.

Hardware target: RTX 5070, 12 GB → batch size 1 at 640². Which is precisely why the batch-size-1 BatchNorm swap bites at load time (section 6).

11 What to take away

The ideas here transfer well beyond room photos.

Reframe the impossible half

“Segment the hidden wall” has no solution. “Remove the sofa, then segment” does. Move the hard part to a model that is good at it.

Ask what a model promises not to do

Imagen's guarantee about untouched pixels mattered more than its image quality — it is what removed an entire re-alignment step.

Design around blind spots

Not around error rates. Two models that fail on the same inputs are one model with extra latency.

Measure where the method acts

Whole-image IoU 0.98 hid a band agreement of 0.74. Scope the metric to the region you are responsible for.

A checkpoint is good only for its job

Same weights: fine inside a box, catastrophic on a full image. Re-benchmark on the downstream task.

Share preprocessing by import

One `wall_trimap()` is the only reason the SDMatte/DiffMatte comparison can be trusted.

Hide latency you cannot remove

MoGe starts first, is collected last, and costs roughly nothing.

Keep your ablations

Every variant notebook is a one-variable experiment, and the ADRs record what each one proved.

OPEN RISKS, STATED PLAINLY

VRAM — four heavy nets on a 12 GB card may need sequential loading. **DINO false positives** — mirror ↔ window needs per-deployment thresholds. **Synthetic→real gap** — quantified by the 0.34 regression. **Flat unnamed wall items** — the acknowledged residual miss. **Cost and non-determinism** — Imagen is billed and stochastic; do not expect bit-reproducible masks across runs.

Deeper written reference: docs/SUMMARY.md · design record: docs/pipeline-design.md, docs/glossary.md, docs/adr/0001-0008.